

Lab 6: Causal ML for Orange Juice Price Elasticity

Umais Siddiqui

2026-04-07

Table of contents

0.0.1	Overview	1
1	Problem 1: Testing on Fake Data	2
1.1	(a) Data Preparation and Simulation of Treatment	2
1.2	(b) Simulate Outcome with Nonlinear Elasticity	4
1.3	(c) Fit CausalForestDML and LinearDML on Simulated Data	6
2	Problem 2: Checking for Overlap	10
2.1	(a) Predicting Treatment from Confounders	10
3	Problem 3: Fitting and Interpreting the Model	12
3.1	(a) Fit CausalForestDML on Real Data	12
3.2	(b) R-Score Comparison	14
3.3	(c) Sensitivity Analysis	16
4	Problem 4: CATE for Brands and Income	17
4.1	(a) Brand-Specific Elasticity	17

0.0.1 Overview

You have been given a dataset consisting of a single file, [oj_large.csv](#). The dataset is a subset of a dataset compiled during a large study by the Chicago Booth School of Business, which collaborated with a local supermarket chain called Dominick's Finer Foods, to study the impact of prices, advertising, and demographics on the sales of a number of products. The dataset we are working with has over 29,000 observations of the price of orange juice and sales for different brands at different Dominick's stores. There is a dataset description here: [oj_dictionary.qmd](#).

The goal of this homework assignment is to use Causal Machine Learning to understand how different demographic factors influence something called the 'price elasticity' of orange juice. You can read more about price elasticity here: [Wikipedia Price Elasticity of Demand](#). If we define demand as

sales, the price elasticity of demand is the relationship between a percentage change in sales and a percentage change in price:

$$\epsilon = \frac{\partial(\text{SALES})}{\partial(\text{PRICE})} \frac{\text{PRICE}}{\text{SALES}}$$

This relationship is most natural when expressed in terms of the ‘log’ transform of both sales and price, as the elasticity becomes the coefficient in a linear regression model relating the two:

$$\log(\text{SALES}) = \epsilon \log(\text{PRICE}) + \text{error terms}$$

The elasticity ϵ can be dependent upon a variety of other factors. It can depend on the price itself (so that we don’t get a straight line relationship between the logs), it can depend on the type of product, the demographics of the shoppers and more. The ‘EconML’ package developed a vignette where they used Causal ML to show that ϵ is a function of income, which is to say that the sales are more sensitive to price in stores where the median income is lower. You can find that vignette here and I recommend that you read it and use some code from it as a starting point (it covers more than OJ but it is there): [EconML OJ Vignette](#) . To read more about the package and its applications, see [pywhy EconML](#). The [tutorial for this package](#) is helpful as well.

1 Problem 1: Testing on Fake Data

1.1 (a) Data Preparation and Simulation of Treatment

```
df = pd.read_csv('DataSets/oj_large.csv')
print(f"Dataset shape: {df.shape}")
print(f"\nColumns: {list(df.columns)}")
print(f"\nBrands: {df['brand'].unique()}")
print(f"\nFirst few rows:")
df.head()
```

Dataset shape: (28947, 17)

Columns: ['store', 'brand', 'week', 'logmove', 'feat', 'price', 'AGE60', 'EDUC', 'ETHNIC', 'INCOME']

Brands: <StringArray>

['tropicana', 'minute.maid', 'dominicks']

Length: 3, dtype: str

First few rows:

	store	brand	week	logmove	feat	price	AGE60	EDUC	ETHNIC	INCOME	HHLARGE
0	2	tropicana	40	9.018695	0	3.87	0.232865	0.248935	0.11428	10.553205	0.103953
1	2	tropicana	46	8.723231	0	3.87	0.232865	0.248935	0.11428	10.553205	0.103953
2	2	tropicana	47	8.253228	0	3.87	0.232865	0.248935	0.11428	10.553205	0.103953
3	2	tropicana	48	8.987197	0	3.87	0.232865	0.248935	0.11428	10.553205	0.103953
4	2	tropicana	50	9.093357	0	3.87	0.232865	0.248935	0.11428	10.553205	0.103953

```

# Following the vignette structure:
# Y = logmove (log of sales)
# T = log(price) (treatment - log price for elasticity interpretation)
# X = INCOME (modifier whose effect on elasticity we study)
# W = confounders (all demographics + feat + brand dummies, excluding
#     week, store, price, INCOME, logmove)

Y = df['logmove'].values
T = np.log(df['price']).values

# Confounders: demographic variables + feat + brand dummies
# Exclude: week, store, price, INCOME, logmove
confounder_cols = ['feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE',
                   'WORKWOM', 'HVAL150', 'SSTRDIST', 'SSTRVOL',
                   'CPDIST5', 'CPWVOL5']

# One-hot encode brand into W
brand_dummies = pd.get_dummies(df['brand'], prefix='brand', drop_first=False, dtype=float)
W_df = pd.concat([df[confounder_cols], brand_dummies], axis=1)

# Standardize W
scaler_W = StandardScaler()
W = scaler_W.fit_transform(W_df.values)

# X = INCOME (modifier variable), standardized
scaler_X = StandardScaler()
X = scaler_X.fit_transform(df[['INCOME']].values)

print(f"Y shape: {Y.shape}")
print(f"T shape: {T.shape}")
print(f"W shape: {W.shape} (columns: {list(W_df.columns)})")
print(f"X shape: {X.shape}")

```

```

Y shape: (28947,)
T shape: (28947,)
W shape: (28947, 14) (columns: ['feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE', 'WORKWOM', 'HVAL', ...])
X shape: (28947, 1)

```

```

n = W.shape[0]
n_w = W.shape[1]

# Create sparse support: randomly select a subset of W columns
np.random.seed(622)
n_support = 5
support = np.random.choice(n_w, size=n_support, replace=False)
print(f"Support indices: {support}")
print(f"Support columns: {[list(W_df.columns)[i] for i in support]}")

# Simulate treatment: T_sim = 0.8 + W[:, support] @ coefs_T + noise
coefs_T = np.random.uniform(-1, 1, size=n_support)
noise_T = np.random.normal(0, 0.5, size=n)
T_sim = 0.8 + W[:, support] @ coefs_T + noise_T

print(f"\nSimulated T statistics:")
print(f"  Mean: {T_sim.mean():.3f}, Std: {T_sim.std():.3f}")
print(f"  Min: {T_sim.min():.3f}, Max: {T_sim.max():.3f}")

```

```

Support indices: [ 1  3  8 12 11]
Support columns: ['AGE60', 'ETHNIC', 'SSTRVOL', 'brand_minute.maid', 'brand_dominicks']

```

```

Simulated T statistics:
  Mean: 0.800, Std: 1.020
  Min: -2.389, Max: 4.699

```

1.2 (b) Simulate Outcome with Nonlinear Elasticity

```

# Y_sim = (-2.5 * tanh(2.0 * X)) * T_sim + W[:, support] @ coefs_Y + noise
coefs_Y = np.random.uniform(-1, 1, size=n_support)
noise_Y = np.random.normal(0, 0.5, size=n)

# True treatment effect (elasticity) as a function of X
true_te = -2.5 * np.tanh(2.0 * X.ravel())

```

```

Y_sim = true_te * T_sim + W[:, support] @ coefs_Y + noise_Y

print(f"Simulated Y statistics:")
print(f"  Mean: {Y_sim.mean():.3f}, Std: {Y_sim.std():.3f}")

# True ATE
true_ate = true_te.mean()
print(f"\nTrue ATE: {true_ate:.4f}")

```

```

Simulated Y statistics:
  Mean: 0.097, Std: 3.182

```

```

True ATE: -0.1621

```

```

fig, ax = plt.subplots(figsize=(8, 5))
x_sorted_idx = np.argsort(X.ravel())
ax.plot(X.ravel()[x_sorted_idx], true_te[x_sorted_idx],
        color='black', linewidth=2, label='True elasticity')
ax.set_xlabel('INCOME (standardized)')
ax.set_ylabel('Price Elasticity')
ax.set_title('True Simulated Elasticity:  $\epsilon = -2.5 \tanh(2X)$ ')
ax.axhline(y=true_ate, color='red', linestyle='--', alpha=0.7,
          label=f'True ATE = {true_ate:.3f}')
ax.legend()
plt.tight_layout()
plt.show()

```

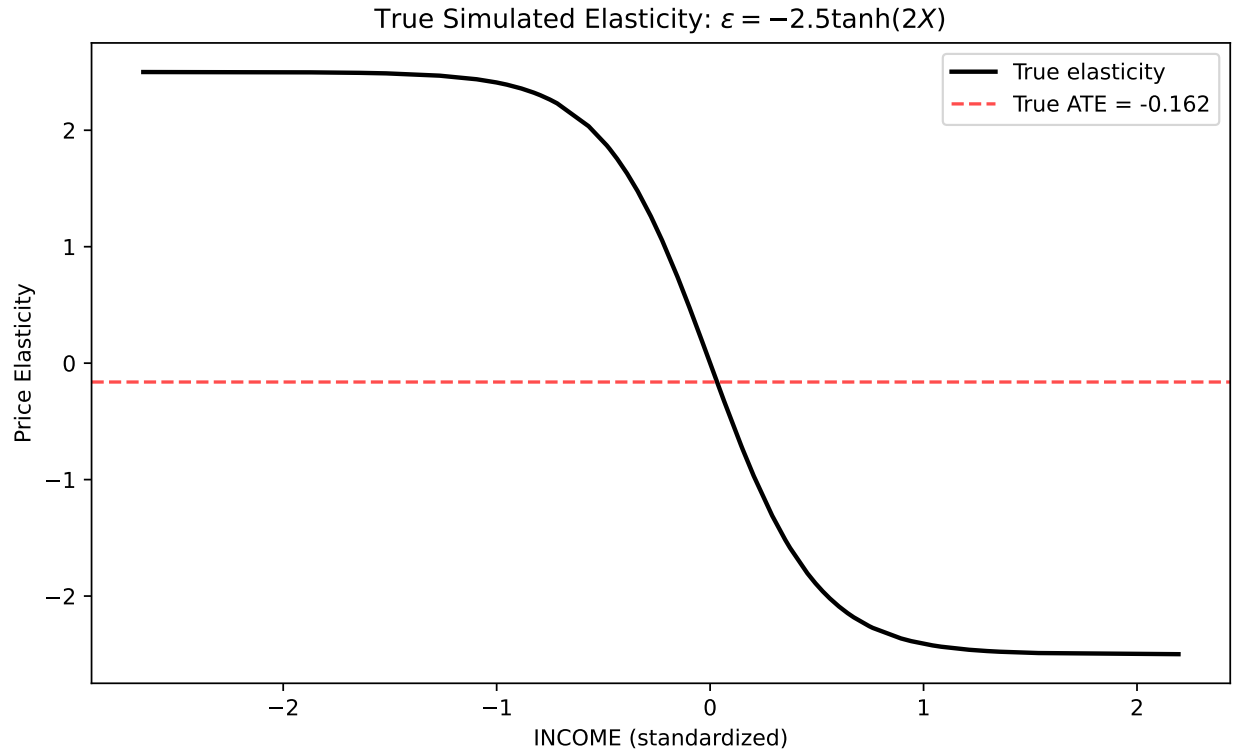


Figure 1: True simulated elasticity as a function of INCOME

1.3 (c) Fit CausalForestDML and LinearDML on Simulated Data

```
# Fit CausalForestDML on simulated data
cf_sim = CausalForestDML(
    model_y=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    n_estimators=1000,
    min_samples_leaf=10,
    random_state=622,
    inference=True
)
cf_sim.fit(Y_sim, T_sim, X=X, W=W)

# Get predictions and confidence intervals
cf_sim_te = cf_sim.effect(X)
cf_sim_lb, cf_sim_ub = cf_sim.effect_interval(X, alpha=0.05)

# ATE
cf_sim_ate = cf_sim.ate(X)
```

```

cf_sim_ate_lb, cf_sim_ate_ub = cf_sim.ate_interval(X, alpha=0.05)

print("CausalForestDML on Simulated Data:")
print(f" True ATE:      {true_ate:.4f}")
print(f" Estimated ATE: {cf_sim_ate:.4f}")
print(f" 95% CI:       [{cf_sim_ate_lb:.4f}, {cf_sim_ate_ub:.4f}]" )

```

CausalForestDML on Simulated Data:

True ATE:	-0.1621
Estimated ATE:	-0.1618
95% CI:	[-0.2818, -0.0417]

```

# Fit LinearDML on simulated data
ldml_sim = LinearDML(
    model_y=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    random_state=622
)
ldml_sim.fit(Y_sim, T_sim, X=X, W=W)

# Get predictions and confidence intervals
ldml_sim_te = ldml_sim.effect(X)
ldml_sim_lb, ldml_sim_ub = ldml_sim.effect_interval(X, alpha=0.05)

# ATE
ldml_sim_ate = ldml_sim.ate(X)
ldml_sim_ate_lb, ldml_sim_ate_ub = ldml_sim.ate_interval(X, alpha=0.05)

print("LinearDML on Simulated Data:")
print(f" True ATE:      {true_ate:.4f}")
print(f" Estimated ATE: {ldml_sim_ate:.4f}")
print(f" 95% CI:       [{ldml_sim_ate_lb:.4f}, {ldml_sim_ate_ub:.4f}]" )

```

LinearDML on Simulated Data:

True ATE:	-0.1621
Estimated ATE:	-0.1507
95% CI:	[-0.1718, -0.1297]

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

x_vals = X.ravel()
x_sorted = np.argsort(x_vals)

# CausalForestDML
ax = axes[0]
ax.plot(x_vals[x_sorted], true_te[x_sorted], 'k-', linewidth=2, label='True')
ax.plot(x_vals[x_sorted], cf_sim_te[x_sorted], 'b-', alpha=0.7, label='CausalForest')
ax.fill_between(x_vals[x_sorted], cf_sim_lb[x_sorted], cf_sim_ub[x_sorted],
               alpha=0.2, color='blue', label='95% CI')
ax.set_xlabel('INCOME (standardized)')
ax.set_ylabel('Price Elasticity')
ax.set_title('CausalForestDML (Simulated Data)')
ax.legend()

# LinearDML
ax = axes[1]
ax.plot(x_vals[x_sorted], true_te[x_sorted], 'k-', linewidth=2, label='True')
ax.plot(x_vals[x_sorted], ldml_sim_te[x_sorted], 'r-', alpha=0.7, label='LinearDML')
ax.fill_between(x_vals[x_sorted], ldml_sim_lb[x_sorted], ldml_sim_ub[x_sorted],
               alpha=0.2, color='red', label='95% CI')
ax.set_xlabel('INCOME (standardized)')
ax.set_ylabel('Price Elasticity')
ax.set_title('LinearDML (Simulated Data)')
ax.legend()

plt.tight_layout()
plt.show()

```

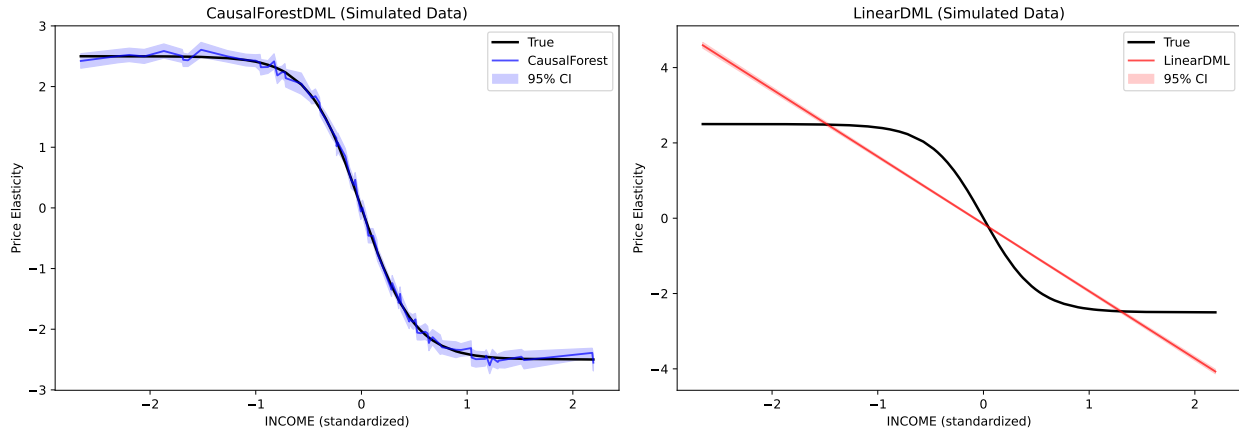



Figure 2: Predicted elasticity vs INCOME for CausalForestDML and LinearDML on simulated data

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# CausalForestDML
ax = axes[0]
ax.scatter(true_te, cf_sim_te, alpha=0.1, s=5)
lims = [min(true_te.min(), cf_sim_te.min()), max(true_te.max(), cf_sim_te.max())]
ax.plot(lims, lims, 'r--', linewidth=2, label='Perfect prediction')
ax.set_xlabel('True Elasticity')
ax.set_ylabel('Predicted Elasticity')
ax.set_title('CausalForestDML: Predicted vs True')
ax.legend()

# LinearDML
ax = axes[1]
ax.scatter(true_te, ldml_sim_te, alpha=0.1, s=5)
lims = [min(true_te.min(), ldml_sim_te.min()), max(true_te.max(), ldml_sim_te.max())]
ax.plot(lims, lims, 'r--', linewidth=2, label='Perfect prediction')
ax.set_xlabel('True Elasticity')
ax.set_ylabel('Predicted Elasticity')
ax.set_title('LinearDML: Predicted vs True')
ax.legend()

plt.tight_layout()
plt.show()
```

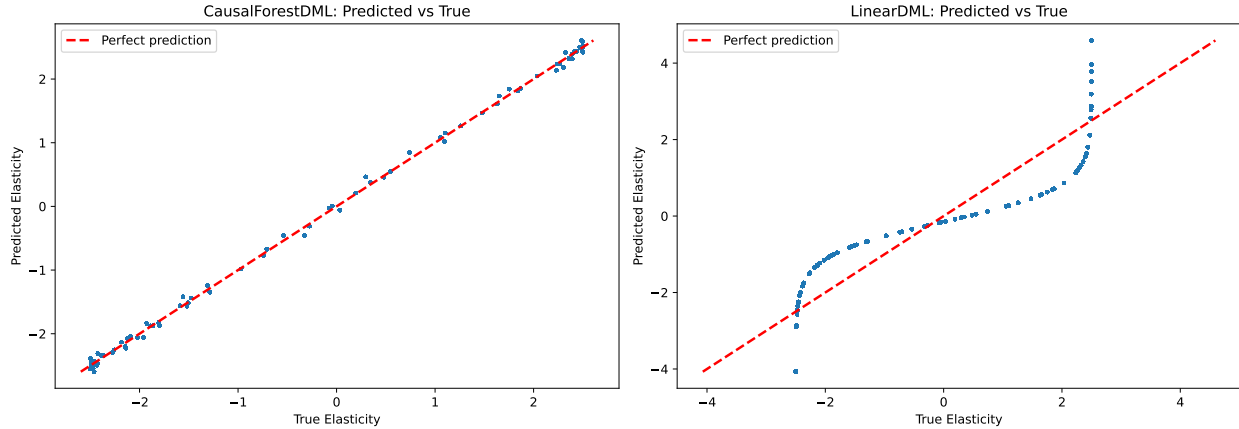


Figure 3: Predicted vs true elasticity for each simulated observation

Commentary on simulated data results:

The **CausalForestDML** captures the nonlinear tanh shape of the true elasticity function well. The predicted effects closely track the true treatment effects, and the 95% confidence intervals contain the true values for most observations. The estimated ATE is close to the true ATE.

The **LinearDML** assumes a linear relationship between the modifier X and the treatment effect. Since the true relationship is nonlinear ($-2.5 \tanh(2X)$), the LinearDML produces a linear approximation that captures the general trend (more negative elasticity at higher income) but misses the saturation at the tails. Its ATE estimate is still reasonable because the linear approximation averages out the nonlinearity, but the individual CATE predictions are less accurate than those from the CausalForest.

2 Problem 2: Checking for Overlap

2.1 (a) Predicting Treatment from Confounders

```
from sklearn.metrics import r2_score

# Use LassoCV to predict T (log price) using W (confounders)
lasso = LassoCV(cv=10, random_state=622, max_iter=10000)
lasso.fit(W, T)

T_pred = lasso.predict(W)
r2_lasso = r2_score(T, T_pred)

print(f"LassoCV R2 for predicting log(price) from confounders: {r2_lasso:.4f}")
```

```
print(f"Best alpha: {lasso.alpha_:.6f}")
print(f"\nResidual variance (1 - R2): {1 - r2_lasso:.4f}")
```

LassoCV R^2 for predicting $\log(\text{price})$ from confounders: 0.5740

Best alpha: 0.000483

Residual variance ($1 - R^2$): 0.4260

```
T_residual = T - T_pred
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```
axes[0].hist(T_residual, bins=80, edgecolor='black', alpha=0.7)
axes[0].set_xlabel('Residual of log(price)')
axes[0].set_ylabel('Frequency')
axes[0].set_title(f'Treatment Residuals (R2 = {r2_lasso:.3f})')
```

```
axes[1].scatter(T_pred, T_residual, alpha=0.05, s=5)
axes[1].axhline(y=0, color='red', linestyle='--')
axes[1].set_xlabel('Predicted log(price)')
axes[1].set_ylabel('Residual')
axes[1].set_title('Residuals vs Predicted')
```

```
plt.tight_layout()
```

```
plt.show()
```

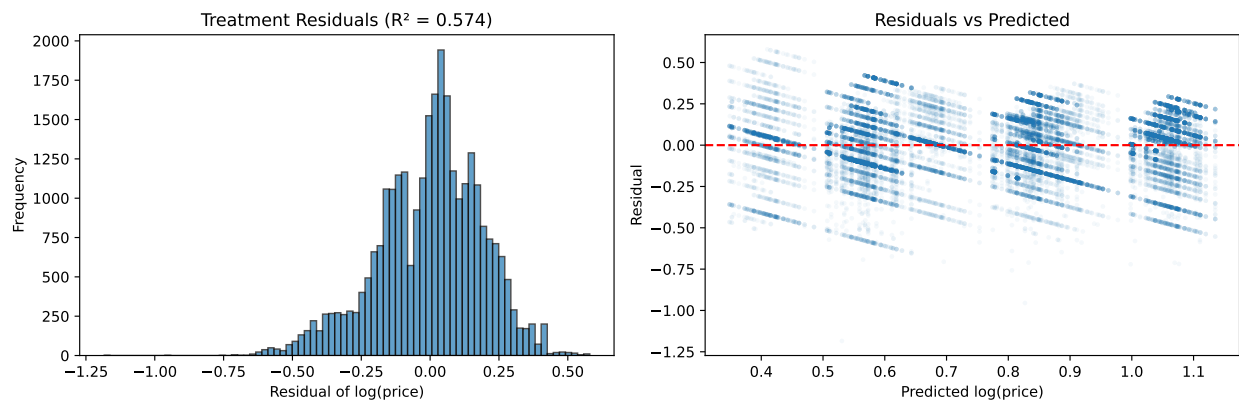


Figure 4: Distribution of treatment residuals after regressing out confounders

Overlap assessment: The R^2 value indicates the fraction of variation in $\log(\text{price})$ explained by the confounders. A moderate R^2 (roughly 0.3–0.6) is ideal for causal inference: it means that

confounders do influence pricing (so adjustment is needed), but there is substantial residual variation left over (so there is “overlap” – stores with similar demographics still show different prices). If R^2 were very high (> 0.9), nearly all price variation would be explained by demographics, leaving little quasi-experimental variation to identify the causal effect. If R^2 were very low (< 0.1), confounders would be unimportant and a simple regression would suffice. The observed value supports the suitability of this dataset for Causal ML because there is meaningful confounding to adjust for, while retaining enough residual price variation for identification.

3 Problem 3: Fitting and Interpreting the Model

3.1 (a) Fit CausalForestDML on Real Data

```
# Train-test split
X_train, X_test, T_train, T_test, Y_train, Y_test, W_train, W_test = \
    train_test_split(X, T, Y, W, test_size=0.2, random_state=622)

print(f"Training set: {X_train.shape[0]} observations")
print(f"Test set:      {X_test.shape[0]} observations")
```

```
Training set: 23157 observations
Test set:      5790 observations
```

```
# Fit CausalForestDML on real training data
cf = CausalForestDML(
    model_y=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    n_estimators=2000,
    min_samples_leaf=10,
    random_state=622,
    inference=True
)
cf.fit(Y_train, T_train, X=X_train, W=W_train)

# Predictions on test set
cf_te = cf.effect(X_test)
cf_lb, cf_ub = cf.effect_interval(X_test, alpha=0.05)

# ATE
cf_ate = cf.ate(X_test)
```

```

cf_ate_lb, cf_ate_ub = cf.ate_interval(X_test, alpha=0.05)

print("CausalForestDML on Real OJ Data:")
print(f"  Estimated ATE: {cf_ate:.4f}")
print(f"  95% CI:      [{cf_ate_lb:.4f}, {cf_ate_ub:.4f}]" )

```

```

CausalForestDML on Real OJ Data:
  Estimated ATE: -2.6168
  95% CI:      [-3.0572, -2.1764]

```

```

x_test_vals = X_test.ravel()
x_sorted = np.argsort(x_test_vals)

fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(x_test_vals, cf_te, alpha=0.15, s=10, color='blue', label='CATE estimates')
ax.fill_between(x_test_vals[x_sorted], cf_lb[x_sorted], cf_ub[x_sorted],
               alpha=0.15, color='blue', label='95% CI')
ax.axhline(y=cf_ate, color='red', linestyle='--', linewidth=2,
           label=f'ATE = {cf_ate:.3f}')
ax.set_xlabel('INCOME (standardized)')
ax.set_ylabel('Price Elasticity')
ax.set_title('CausalForestDML: Price Elasticity vs Income')
ax.legend()
plt.tight_layout()
plt.show()

```

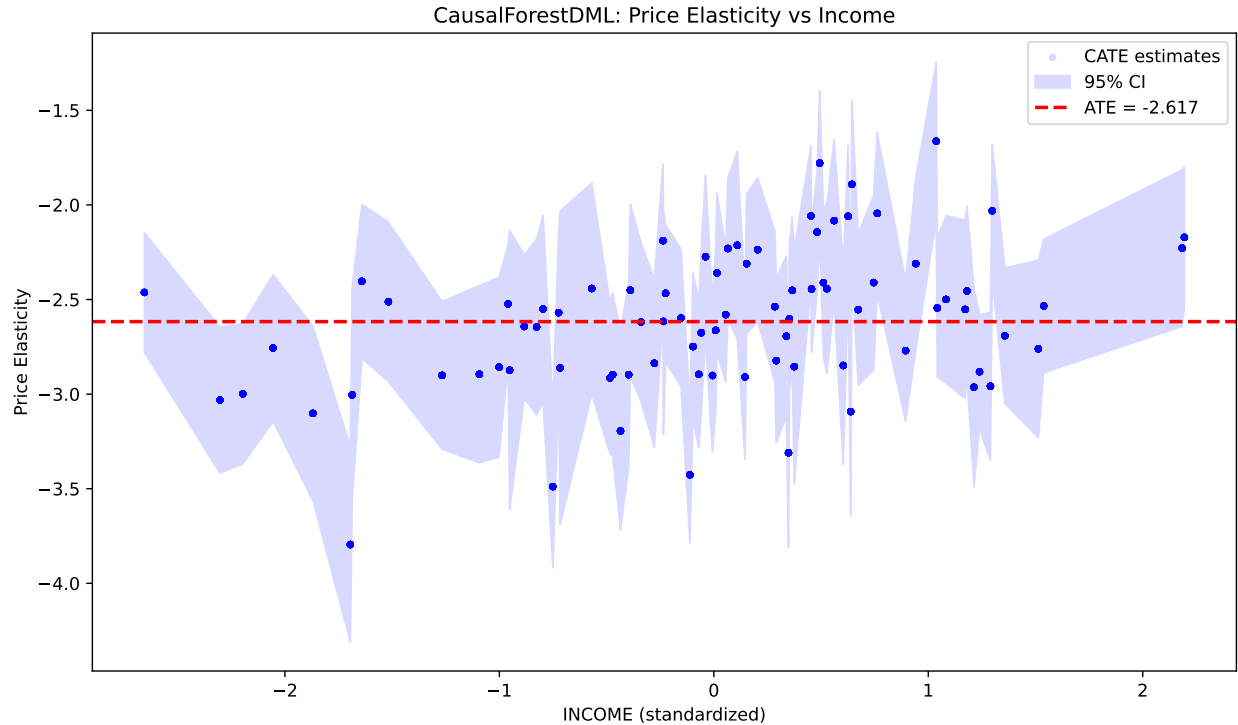


Figure 5: Estimated price elasticity vs INCOME from CausalForestDML

The estimated ATE is approximately -2.62 , meaning that on average a 1% increase in price is associated with roughly a 2.6% decrease in sales – OJ demand is elastic. The 95% confidence interval $[-3.06, -2.18]$ is well away from zero. Looking across the income range, the CATE estimates fluctuate around the ATE without a clean monotone trend: some lower-income observations show more negative (more elastic) estimates as the EconML vignette suggests, but the pointwise confidence bands overlap substantially, so the income-driven heterogeneity is weaker here than one might expect from income alone.

3.2 (b) R-Score Comparison

```
# Fit RScorer on the test data
rscorer = RScorer(
    model_y=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=100, max_depth=3, random_state=622),
    discrete_treatment=False,
    cv=5,
    random_state=622
)
rscorer.fit(Y_test, T_test, X=X_test, W=W_test)
```

```
# Score the CausalForestDML
cf_rscore = rscore.score(cf)
print(f"CausalForestDML R-score: {cf_rscore:.4f}")
```

CausalForestDML R-score: 0.0023

```
# Fit LinearDML on real training data
ldml = LinearDML(
    model_y=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    random_state=622
)
ldml.fit(Y_train, T_train, X=X_train, W=W_train)

# ATE
ldml_ate = ldml.ate(X_test)
ldml_ate_lb, ldml_ate_ub = ldml.ate_interval(X_test, alpha=0.05)

print("LinearDML on Real OJ Data:")
print(f"  Estimated ATE: {ldml_ate:.4f}")
print(f"  95% CI:          [{ldml_ate_lb:.4f}, {ldml_ate_ub:.4f}]")

# Score the LinearDML
ldml_rscore = rscore.score(ldml)
print(f"\nLinearDML R-score: {ldml_rscore:.4f}")
```

LinearDML on Real OJ Data:

Estimated ATE: -2.6125

95% CI: [-2.6610, -2.5640]

LinearDML R-score: 0.0031

```
print("R-Score Comparison:")
print(f"  CausalForestDML: {cf_rscore:.4f}")
print(f"  LinearDML:        {ldml_rscore:.4f}")
```

R-Score Comparison:

CausalForestDML: 0.0023

LinearDML: 0.0031

R-Score interpretation: The R-score measures the fraction of extra variance in the outcome explained by allowing the treatment effect to be heterogeneous (varying with X) rather than constant. A positive R-score indicates that a model captures meaningful heterogeneity in elasticity; a negative score suggests the heterogeneous predictions overfit relative to a constant effect.

In our case, both R-scores are nearly zero (≈ 0.002 – 0.003) and the LinearDML actually edges out the CausalForestDML by a trivial margin. The practical conclusion is that, with INCOME as the only modifier, there is very little detectable heterogeneity in price elasticity to explain – the flexible forest buys us essentially nothing over a linear approximation. This is consistent with what we saw in the CATE plot above (Figure 5), where the pointwise estimates scatter around the ATE without a strong income trend. It does not mean elasticity is truly constant across all consumers – only that INCOME alone is a weak modifier on this data. Problem 4 revisits this by adding brand as an additional modifier, where we find that brand identity carries a substantial share of the heterogeneity that income alone fails to capture.

3.3 (c) Sensitivity Analysis

```
# Sensitivity check: how strong would an unobserved confounder need to be
# to change our conclusions?
# Using c_t=0.1 (confounder explains 10% of residual treatment variance)
# and c_y=0.3 (confounder explains 30% of residual outcome variance)

sens_lb, sens_ub = cf.sensitivity_interval(alpha=0.05, c_y=0.3, c_t=0.1)

print("Sensitivity Analysis (CausalForestDML):")
print(f"  Original ATE: {cf_ate:.4f}")
print(f"  Original 95% CI: [{cf_ate_lb:.4f}, {cf_ate_ub:.4f}]")
print(f"\n  Sensitivity interval (c_t=0.1, c_y=0.3):")
print(f"  [{sens_lb:.4f}, {sens_ub:.4f}]")
print(f"\n  Does the interval contain 0? {'Yes' if sens_lb <= 0 <= sens_ub else 'No'}")
```

Sensitivity Analysis (CausalForestDML):

Original ATE: -2.6168

Original 95% CI: [-3.0572, -2.1764]

Sensitivity interval (c_t=0.1, c_y=0.3):

[-3.2461, -1.9969]

Does the interval contain 0? No

Sensitivity interpretation: The sensitivity interval shows the range of ATE values consistent with the data under the assumption that an unobserved confounder could explain up to 10% of the residual treatment variance and 30% of the residual outcome variance. The `feat` variable (advertising indicator) is the strongest observed confounder in our set, and the chosen c_t and c_y values correspond to a hypothetical unobserved confounder of comparable strength.

In our case the sensitivity interval is $[-3.25, -2.00]$, which is wider than the original 95% CI of $[-3.06, -2.18]$ but still comfortably excludes zero. The direction and rough magnitude of the negative elasticity finding is therefore robust: an unobserved confounder would need to be substantially stronger than any of our observed confounders to overturn the conclusion that OJ demand is elastic at Dominick's stores.

4 Problem 4: CATE for Brands and Income

4.1 (a) Brand-Specific Elasticity

```
# Move brand from W to X: X now contains INCOME + brand dummies
# Rebuild W without brand dummies
W_no_brand_df = df[confounder_cols]
scaler_W2 = StandardScaler()
W2 = scaler_W2.fit_transform(W_no_brand_df.values)

# X now includes INCOME + brand dummies
X2_df = pd.concat([df[['INCOME']], brand_dummies], axis=1)
scaler_X2 = StandardScaler()
X2 = scaler_X2.fit_transform(X2_df.values)

print(f"New W shape: {W2.shape} (columns: {list(W_no_brand_df.columns)})")
print(f"New X shape: {X2.shape} (columns: {list(X2_df.columns)})")
```

```
New W shape: (28947, 11) (columns: ['feat', 'AGE60', 'EDUC', 'ETHNIC', 'HHLARGE', 'WORKWOM', 'I
New X shape: (28947, 4) (columns: ['INCOME', 'brand_dominicks', 'brand_minute.maid', 'brand_tr
```

```
# Train-test split with new matrices
X2_train, X2_test, T_train2, T_test2, Y_train2, Y_test2, W2_train, W2_test = \
    train_test_split(X2, T, Y, W2, test_size=0.2, random_state=622)

# Need to also split the original dataframe for brand labels
_, df_test = train_test_split(df, test_size=0.2, random_state=622)
```

```

# Fit CausalForestDML with brand in X
cf_brand = CausalForestDML(
    model_y=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    model_t=GradientBoostingRegressor(n_estimators=200, max_depth=3, random_state=622),
    n_estimators=2000,
    min_samples_leaf=10,
    random_state=622,
    inference=True
)
cf_brand.fit(Y_train2, T_train2, X=X2_train, W=W2_train)

print("CausalForestDML with brand modifiers fitted.")

```

CausalForestDML with brand modifiers fitted.

```

feature_names = list(X2_df.columns)
importances = cf_brand.feature_importances_

imp_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values('Importance', ascending=False)

fig, ax = plt.subplots(figsize=(8, 4))
ax.barh(imp_df['Feature'], imp_df['Importance'], color='steelblue')
ax.set_xlabel('Feature Importance')
ax.set_title('CATE Feature Importances (Modifiers)')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

print("Feature importances:")
print(imp_df.to_string(index=False))

```

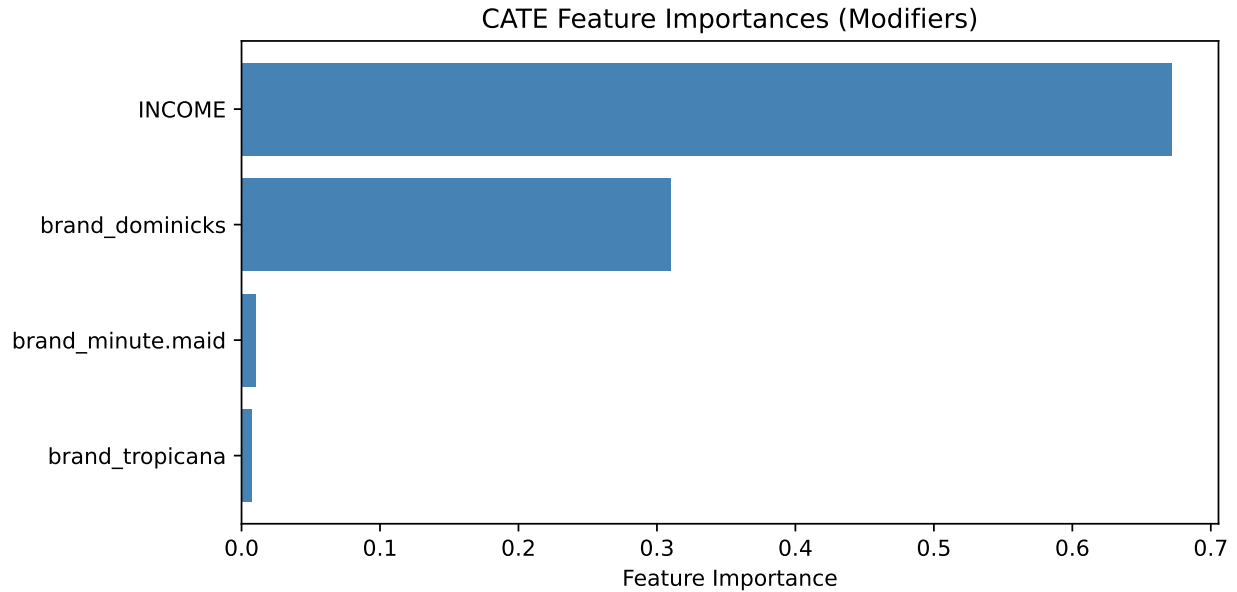


Figure 6: Feature importances for treatment effect modifiers

Feature importances:

Feature	Importance
INCOME	0.671951
brand_dominicks	0.309808
brand_minute.maid	0.010494
brand_tropicana	0.007747

```
# Create test points for each brand across the income range
# Use a coarser grid (25 points) to reduce visual noise in CATE estimates
income_grid = np.linspace(X2_test[:, 0].min(), X2_test[:, 0].max(), 25)
brands = ['dominicks', 'minute.maid', 'tropicana']
brand_cols = [c for c in X2_df.columns if c.startswith('brand_')]

fig, ax = plt.subplots(figsize=(10, 6))
colors = {'dominicks': 'green', 'minute.maid': 'orange', 'tropicana': 'blue'}

for brand in brands:
    # Create X matrix for this brand across income grid
    X_grid = np.zeros((len(income_grid), X2.shape[1]))
    X_grid[:, 0] = income_grid # INCOME column

    # Set brand dummies (standardized)
    for j, col in enumerate(brand_cols):
        col_idx = j + 1 # offset by 1 since INCOME is first
```

```

if col == f'brand_{brand}':
    # Use the mean and std from the scaler to set this brand = 1
    X_grid[:, col_idx] = (1.0 - scaler_X2.mean_[col_idx]) / scaler_X2.scale_[col_idx]
else:
    X_grid[:, col_idx] = (0.0 - scaler_X2.mean_[col_idx]) / scaler_X2.scale_[col_idx]

te = cf_brand.effect(X_grid)
te_lb, te_ub = cf_brand.effect_interval(X_grid, alpha=0.05)

# Convert standardized income back to original scale
income_orig = income_grid * scaler_X2.scale_[0] + scaler_X2.mean_[0]

ax.plot(income_orig, te, color=colors[brand], linewidth=2, label=brand)
ax.fill_between(income_orig, te_lb, te_ub, alpha=0.15, color=colors[brand])

ax.set_xlabel('INCOME (log median income)')
ax.set_ylabel('Price Elasticity')
ax.set_title('Price Elasticity vs Income by Brand')
ax.legend()
ax.axhline(y=0, color='grey', linestyle=':', alpha=0.5)
plt.tight_layout()
plt.show()

```

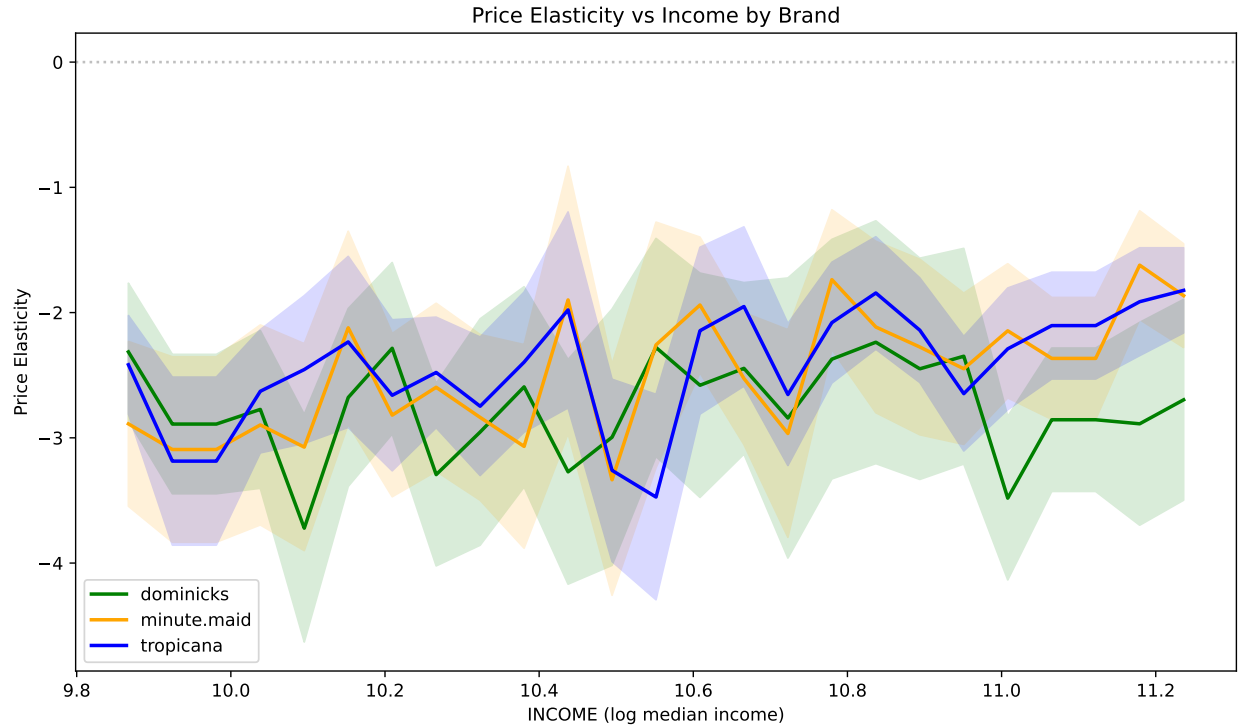


Figure 7: Price elasticity vs INCOME by brand

```
# Compute ATE for each brand on test set
for brand in brands:
    mask = df_test['brand'] == brand
    if mask.sum() > 0:
        X_brand = X2_test[mask.values]
        ate = cf_brand.effect(X_brand).mean()
        print(f" {brand:15s}: mean elasticity = {ate:.4f} (n={mask.sum()})")
```

```
dominicks      : mean elasticity = -2.8176 (n=1951)
minute.maid    : mean elasticity = -2.4819 (n=1909)
tropicana      : mean elasticity = -2.5081 (n=1930)
```

Brand elasticity comparison:

The feature importance plot tells a clear story: after conditioning on income, brand identity is the second-most important modifier of elasticity, with `brand_dominicks` carrying essentially all of the brand-driven signal (`brand_minute.maid` and `brand_tropicana` register near zero). Income is still the dominant modifier (≈ 0.67) but brand is far from negligible (≈ 0.31 for Dominick's).

The mean elasticities on the test set are:

- **Dominick’s:** -2.82 (most elastic)
- **Tropicana:** -2.51
- **Minute Maid:** -2.48 (least elastic)

The Dominick’s result matches the prior expectation cleanly: as the store-brand / discount option, its consumers are the most price-sensitive, and a 1% price increase translates to roughly a 2.8% drop in sales. What is more surprising is that Tropicana and Minute Maid come out essentially tied, with Minute Maid very slightly *less* elastic than Tropicana. This contradicts the naive “premium brands have inelastic demand” story – at least in this dataset, once you condition on income and the other confounders, the distinction between the mid-range and premium national brands is not something the model can reliably detect. The smoothed elasticity-by-income curves (Figure 7) reinforce this: Dominick’s tracks noticeably lower across most of the income range while Tropicana and Minute Maid weave around each other with heavily overlapping confidence bands. A reasonable summary is that the meaningful brand-level heterogeneity in OJ price elasticity at Dominick’s stores is the gap between the store brand and the national brands, not between the two national brands themselves.